

Lecture Unit 4.2

Point-to-point communication

Point-to-point communication

- With MPI (Message Passing Interface) processes communicate and co-ordinate by passing messages
- One type of communication is point-to-point communication
- Point-to-point communication takes place between a pair of processes in a communicator
- The most basic functions for sending and receiving point-to-point messages are `MPI_Send` and `MPI_Recv`

MPI_Send

- `MPI_Send`: send a message to another process
- Describes data to be sent and connection to be made
- `int MPI_Send(void *buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm)`

Number of data items not bytes → **count**: number of items to send

Defines destination process → **dest**: which rank to send to

buf: send buffer

Defines message length → **datatype**: type of data

tag: user defined label for this communication

comm: communicator e.g. `MPI_COMM_WORLD`

MPI C data types

MPI_CHAR	signed char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double

Point-to-point: MPI_Recv

- `MPI_Recv`: receive a message from another process
- Describes data to be received and connection to be made
- `int MPI_Recv(void *buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status *status)`

buf: receive buffer

count: number of items to receive

datatype: type of data

source: where to receive from

tag: user defined label for this communication

comm: communicator e.g. `MPI_COMM_WORLD`

status: rank and tag received from

Defines message length

Defines source process

point-to-point.c

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[]){

    int myRank;           // Process rank
    float number;         // The number to send/receive
    const int tag = 999;  // User defined tag for the message
    MPI_Status status;    // MPI_Status is defined in mpi.h

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &myRank);

    /* Rank 0 process sends the number */
    if (myRank==0){
        number=3.141;
        printf("I am process %d sending the number %f\n", myRank, number);
        MPI_Send(&number, 1, MPI_FLOAT, 1, tag, MPI_COMM_WORLD);
    }
    /* Rank 1 process receives the number */
    else if(myRank==1){
        MPI_Recv(&number, 1, MPI_FLOAT, 0, tag, MPI_COMM_WORLD, &status);
        printf("I am process %d the number is %f\n", myRank, number);
    }

    MPI_Finalize();

    return 0;
}
```

point-to-point.c

```
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ module load OpenMPI/4.1.5-GCC-12.3.0
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ mpicc -o pointtopoint point-to-point.c
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ ls
bcast.c gather.c gather_malloc.c hello_mpi hello_mpi.c pointtopoint point-to-point.c
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ mpirun -np 2 pointtopoint
I am process 0 sending the number 3.141000
I am process 1 the number is 3.141000
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ mpirun -np 4 pointtopoint
I am process 0 sending the number 3.141000
I am process 1 the number is 3.141000
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ mpirun -np 3 pointtopoint
I am process 0 sending the number 3.141000
I am process 1 the number is 3.141000
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ mpirun -np 8 pointtopoint
I am process 0 sending the number 3.141000
I am process 1 the number is 3.141000
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ |
```

Unit Summary

- MPI Point-to-point communication takes place between a pair of processes in a communicator
- Basic point-to-point send and receive functions:
 - `MPI_Send`
 - `MPI_Recv`
- Message lengths are specified in terms of MPI data types (not bytes)